

Institute of Business Administration, Karachi
CSE141 — Introduction to Programming — Fall 2026

HOMEWORK 01

Decisions, Iteration, and Programs that Carry State

Released	Monday, September 7, 2026, 9:00 AM PKT	**Total Points**	100
Due	Sunday, September 27, 2026, 11:55 PM PKT	**Questions**	5 × 20 points
Submission	LMS — five separate `.cpp` files	**Language**	C++ (g++)
Topics	Weeks 1–4	**Applies to**	All sections

Instructions

Permitted Material

This homework covers only the material presented through **Week 4**. You may use any of the following:

- Variables and built-in data types (`int`, `double`, `char`, `bool`)
- Arithmetic and comparison operators, and type conversion
- `if`, `if-else`, and nested `if-else`
- `switch`
- `while`, `for`, and `do-while` loops, including nested loops
- `break` and `continue`
- For Question 4 only, `sleep(1)` from `<unistd.h>` may be used as an optional visual delay while demonstrating the clock. It is not required for grading.

Not Permitted

You may **not** use arrays, user-defined functions, recursion, pointers, strings, `std::string`, STL containers such as `vector`, library helpers such as `abs()` or `sqrt()`, or any topic that has not yet been covered in class. A program that produces correct output using prohibited material will receive **zero** for that question, regardless of correctness. Every value your program tracks must be held in an ordinary scalar variable, even when a question involves several related quantities.

Submission Requirements

1. Submit **five separate** `.cpp` files, named exactly as specified in each question. Do not archive them into a single file unless the LMS requires it.

2. Every program must compile with g++ without errors. A program that does not compile receives zero for that question.
3. Include a comment block at the top of each file with your **name**, **ERP ID**, and **section**.
4. Upload to the LMS before the deadline. Submissions sent by email are not accepted.

Grading

Each question is worth **20 points**, distributed as follows: correctness of output and logic (12), correct handling of the specified edge and boundary cases (4), and code quality — meaningful variable names, consistent indentation, and clear prompts (4).

Academic Integrity

All submitted work must be your own. You may discuss general concepts with classmates, but the code you submit must be written by you alone. Sharing code, copying from any other student, or submitting code generated by an AI tool as your own is a violation of the Institute's academic integrity policy and will be reported. You may be asked to explain any part of your submission in person; being unable to do so will be treated as evidence of plagiarism.

Late Policy

Submissions received after the deadline are penalized **10% per day**, up to a maximum of three days. After 72 hours, the submission will not be graded. Extensions are granted only for documented medical or family emergencies.

A Note on Output Formatting

Match the sample output shown in each question as closely as you reasonably can. Where a question does not specify exact wording, use your own clear prompts — but the structure, ordering, and values of the output must match. Test each program with several inputs, including the boundary cases explicitly described in the question.

Question 1 — Kaprekar's Constant Chase

File: `kaprekar.cpp` **Marks:** 20

In 1949, the mathematician D. R. Kaprekar described a curious property of four-digit numbers. Take any four-digit number whose digits are not all identical, rearrange its digits to form the largest and the smallest numbers possible, and subtract the smaller from the larger. Repeat the process on the result. No matter which number you start from, you always arrive at **6174** within a handful of steps — and once you reach it, the process produces 6174 forever after. This value is known as Kaprekar's constant.

Write a program that reads a four-digit number from the user and plays this process out, printing every step.

Input Validation

If the number entered is outside the range 1000 to 9999, or if all four of its digits are identical (for example 1111 or 7777), reject it and ask again. Numbers with four identical digits produce a difference of zero and can never reach 6174.

Requirements

1. Extract the four digits numerically using `/` and `%`. Do not convert the number to a string.
2. Arrange the digits into descending order using only comparisons and swaps between digit variables, with a temporary variable. The largest arrangement is the digits in descending order; the smallest is the same digits in ascending order.
3. Subtract the smaller arrangement from the larger one, print the step, and repeat with the result until it equals 6174.
4. Print the total number of steps taken.

Sample Output

```
Enter a four-digit number:  3524
Step 1:  5432 - 2345 = 3087
Step 2:  8730 - 0378 = 8352
Step 3:  8532 - 2358 = 6174
Reached Kaprekar's constant in 3 steps.
```

Additional Sample Output — Validation and Leading Zeros

```
Enter a four-digit number:  7777
Invalid input.  Enter a four-digit number whose digits are not all identical:  1000
Step 1:  1000 - 0001 = 0999
Step 2:  9990 - 0999 = 8991
Step 3:  9981 - 1899 = 8082
Step 4:  8820 - 0288 = 8532
Step 5:  8532 - 2358 = 6174
Reached Kaprekar's constant in 5 steps.
```

Hints

- Extract the digits with `d1 = n / 1000`, `d2 = (n / 100) % 10`, `d3 = (n / 10) % 10`, and `d4 = n % 10`.
- To order four variables without an array, compare and swap adjacent pairs, the way you would sort four playing cards by hand. Three passes over the three adjacent pairs is enough to guarantee full ordering. Writing the passes as a small `for` loop is good practice, but writing the swaps out directly is also acceptable.
- The smallest arrangement is simply the sorted digits read in the opposite order, so you do not need to sort twice.
- A leading zero in the smaller number is expected — 0378 is just the value 378 when you subtract. Print the leading zeros yourself so that the step lines line up as shown.
- Guard the loop with a safety limit, such as stopping after 15 steps, in case of an error in your ordering logic. A correct implementation always finishes within seven steps.

Question 2 — Wi-Fi Signal Map

File: `wifiMap.cpp` **Marks:** 20

A Wi-Fi router sits at the exact center of a square room, which is represented as an $n \times n$ grid of cells. The strength of the signal in a cell depends on how far that cell is from the router, measured as the **Manhattan distance** — the number of steps needed to reach the router moving only horizontally and vertically. Write a program that reads n and prints the resulting signal map.

Input Validation

The value of n must be an **odd** integer from 3 to 15, so that the grid has an exact center cell. If the user enters an invalid value, **do not round it and do not terminate the program**; print an appropriate message and keep asking until a valid value is entered. Check the range first: if $n < 3$ or $n > 15$, report that it must be between 3 and 15. Otherwise, if it is even, report that it must be odd.

Requirements

1. Generate the entire grid using nested loops. Do not hard-code any row, and do not print rows individually.
2. The cell containing the router prints `0`. Each step away from the router, horizontally or vertically, increases the printed value by 1.
3. Do **not** use `abs()` or any other library function. Compute the horizontal and vertical distances using `if` statements to decide which value to subtract from which.
4. Separate the values in a row by a single space, and print each row on its own line.

Sample Output

```
Enter grid size n (odd, 3 to 15):  8
n must be an odd number.  Please try again.
Enter grid size n (odd, 3 to 15):  7
6 5 4 3 4 5 6
5 4 3 2 3 4 5
4 3 2 1 2 3 4
```

```
3 2 1 0 1 2 3
4 3 2 1 2 3 4
5 4 3 2 3 4 5
6 5 4 3 4 5 6
```

Additional Sample Output — Range and Smallest Valid Grid

```
Enter grid size n (odd, 3 to 15):  16
n must be between 3 and 15.  Please try again.
Enter grid size n (odd, 3 to 15):  4
n must be an odd number.  Please try again.
Enter grid size n (odd, 3 to 15):  3
2 1 2
1 0 1
2 1 2
```

Hints

- For an odd n, the center row and center column are both at index `n / 2`, using integer division.
- Use an outer loop for the row index and an inner loop for the column index. The value in a cell is the row distance plus the column distance.
- To find a distance without `abs()`: if the index is greater than the center, subtract the center from the index; otherwise subtract the index from the center.
- A number is odd when `n % 2 != 0`. Check this together with the range check inside your validation loop.

Question 3 — Run-Length Stream Compressor

File: `runLength.cpp` **Marks:** 20

Run-length encoding is a simple compression technique in which a run of consecutive identical values is stored once, together with a count of how many times it repeated. A stream such as 5 5 5 2 2 9 9 9 1 1 compresses to four entries instead of eleven. Write a program that compresses a stream of integers as the user types them in.

The essential constraint is that you cannot see the whole stream at once. Because arrays are not permitted, you must process each value **as it arrives**, keeping only the value currently being counted and the length of its run.

A value **may appear again in a later run**. Runs are determined only by **consecutive** equal values. For example, in `5 5 5 6 6 5 5`, the two groups of 5s are separate runs because the 6s occur between them.

Requirements

1. Read integers one at a time until the user enters **-1**, which terminates the stream and is not part of the data.
2. Count consecutive occurrences of the same value. Each time the incoming value differs from the one being counted, print the completed run and begin a new one. If the same value appears

again later after one or more different values, it starts a **new run**; it is not merged with an earlier run.

3. Print each run in the format `[count x value]`, with all runs on a single line separated by spaces.
4. Do not forget the final run: when -1 arrives, the run in progress has not yet been printed.
5. After the compressed output, print the total number of values read, the number of compressed entries produced, and the percentage reduction, given to two decimal places.
6. If the very first value entered is -1, the stream is empty. Print a message saying so and end the program without attempting any statistics.

Sample Output

```
Enter integers, -1 to end:  5 5 5 2 2 9 9 9 9 1 1 -1
Compressed output:
[3 x 5] [2 x 2] [4 x 9] [2 x 1]
Original values     :  11
Compressed entries  :  4
Reduction           :  63.64%
```

Additional Sample Output — Same Value in Separate Runs

```
Enter integers, -1 to end:  5 5 5 5 6 6 5 5 5 5 -1
Compressed output: [4 x 5] [2 x 6] [4 x 5]
Original values     :  10
Compressed entries  :  3
Reduction           :  70.00%
```

Additional Sample Output — Empty Stream

```
Enter integers, -1 to end:  -1
The stream is empty.
```

Hints

- You need very few variables: the value currently being counted, the length of the current run, the value just read, and two counters for the totals.
- The first value read is a special case, because there is no previous run to compare against. Handle it before the main loop, or use a flag to mark the first iteration.
- The reduction is computed as (original − compressed) / original, expressed as a percentage. Both counters are integers, so you must convert to `double` before dividing, or the result will truncate to zero.
- Use `cout << fixed << setprecision(2)` to print the percentage to two decimal places. This requires `#include <iomanip>`, which is permitted.

Question 4 — Snooze Clock

File: `snoozeClock.cpp` **Marks:** 20

Write a program that simulates a bedside alarm clock. The clock advances one second at a time from a starting time given by the user, and rings when it reaches the alarm time. The user may then snooze or get up — but only so many times.

Requirements

1. Read the current time as three integers — hour, minute, and second, in 24-hour format — and the alarm time as two integers, hour and minute.
2. Advance the clock one second per iteration. Print nothing while the clock is simply ticking. For a visible real-time demonstration, you may place `sleep(1)` after each clock update; this delay is optional and may be commented out while testing.
3. Use nested `if` statements to handle rollover: 60 seconds become one minute, 60 minutes become one hour, and hour 24 becomes hour 0.
4. When the clock's hour and minute match the alarm's hour and minute, print `BEEP BEEP BEEP! It's HH:MM!` and offer the user a choice using a `switch`: **1) Snooze** or **2) Wake up**.
5. Snoozing adds five minutes to the alarm time, handling the hour rollover if the minutes pass 60, and the clock continues to tick.
6. The user may snooze at most **three** times. On the fourth ring, do not offer the choice — print `No more snoozing - up you get!` and end the simulation.
7. Choosing to wake up ends the simulation immediately with `Good morning! You woke up after N snooze(s).`

Sample Output

```
Enter current time (hh mm ss): 7 58 55
Enter alarm time (hh mm): 8 0
BEEP BEEP BEEP! It's 08:00!
1) Snooze 2) Wake up: 1
Snoozing for five more minutes...
BEEP BEEP BEEP! It's 08:05!
1) Snooze 2) Wake up: 2
Good morning! You woke up after 1 snooze(s).
```

Additional Sample Output — Midnight Rollover

```
Enter current time (hh mm ss): 23 59 58
Enter alarm time (hh mm): 0 0
BEEP BEEP BEEP! It's 00:00!
1) Snooze 2) Wake up: 2
Good morning! You woke up after 0 snooze(s).
```

Hints

- Keep the hour, minute, and second of the running clock in three separate `int` variables, and the alarm hour and minute in two more.
- The rollover checks belong immediately after you increment the seconds. Check `seconds == 60` first; only if that occurred do you check `minutes == 60`, and only then `hours == 24`.

- Compare against the alarm time only when the seconds equal 0. Otherwise the alarm will fire sixty times during the matching minute.
- Print a leading zero for hours and minutes below 10 so that the time reads `08:05` rather than `8:5`.
- Both the wake-up case and the snooze-limit case should leave the loop using `break`.

Question 5 — Mars Rover

File: `marsRover.cpp` **Marks:** 20

A rover has landed on a small square planet and is being driven remotely, one command at a time. The planet's surface is a coordinate grid in which both x and y run from **-5 to 5**, inclusive. The rover begins at the origin $(0, 0)$, facing **North**.

Somewhere on the surface there is an open pit and a hidden treasure. Their locations are fixed and known to your program, but not to the driver. If the rover moves onto the pit, the mission fails; if it reaches the treasure first, the mission succeeds. Fixed locations are used so that every student's program can be tested reproducibly.

Mission control sends single-character commands, which your program reads one at a time:

Command	Meaning
`F`	Move one step forward in the direction the rover is currently facing
`L`	Turn 90 degrees to the left, without moving
`R`	Turn 90 degrees to the right, without moving
`P`	Print the rover's current position and direction
`Q`	End the mission and quit the program

Requirements

1. Maintain the rover's x-coordinate, y-coordinate, and facing direction across the whole session. The direction must be held in a single variable — an `int` coded as 0 = North, 1 = East, 2 = South, 3 = West works well.
2. Moving North increases y; South decreases y; East increases x; West decreases x.
3. **Boundary rule:** if an `F` command would take the rover outside the range -5 to 5 on either axis, the rover must stay exactly where it is and the program must print `Boundary reached.` The direction is unchanged.
4. **Hidden locations:** hard-code the pit at **(-3, 2)** and the treasure at **(2, 3)**, using separate scalar variables for their coordinates. Do not display these coordinates while the mission is in progress.
5. **Pit rule:** if a legal move places the rover exactly on the pit, print `The rover has fallen into a pit. Mission failed.` and end the mission immediately.
6. **Treasure rule:** if a legal move places the rover exactly on the treasure before it reaches the pit, print `Treasure found! Mission successful.` and end the mission immediately.
7. After either success, failure, or early-end via Q, print a final 11×11 map using nested loops. Print `P` for the pit, `T` for the treasure, `R` for the rover if it is on neither special square, and `X` for the rover's final square when it coincides with the pit or treasure. Print `-` for every other cell. Use rows from $y = 5$ down to $y = -5$ so North appears at the top. This final map reveals both hidden locations only after the mission has ended.

8. Turning must wrap around correctly: turning right from West faces North, and turning left from North faces West.
9. A command that is not one of F, L, R, P, or Q is invalid. Print `Unknown command.` and continue reading; an invalid command must not change the rover's state.
10. Accept commands in either upper or lower case. Convert a lowercase letter to uppercase using character arithmetic — subtracting 32 from its character code — rather than testing for both cases everywhere.

Sample Session — Mission Success

```
Command:  F
Command:  F
Command:  F
Command:  R
Command:  F
Command:  F
Treasure found!  Mission successful.
Final map:
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - X - - -
- - P - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
```

Additional Sample Session — Mission Failure

```
Command:  L
Command:  F
Command:  F
Command:  F
Command:  R
Command:  F
Command:  F
The rover has fallen into a pit.  Mission failed.
Final map:
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - T - - -
- - X - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
```

Additional Sample Session — Invalid Command and Quit

```
Command:  Z
Unknown command.
Command:  P
Position:  (0, 0)
Direction:  North
Command:  Q
Mission ended.
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - T - - -
- - P - - - - - - - -
- - - - - - - - - - -
- - - - - R - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
- - - - - - - - - - -
```

Hints

- A `switch` on the command character is the natural structure here, with one case per command and a `default` for unknown input.
- Turning right is the same as adding 1 to the direction code and wrapping with `% 4`. Turning left is adding 3 and wrapping with `% 4`, which avoids a negative result.
- For the `F` command, work out the intended new coordinate first, then check whether it lies within -5 to 5 before committing to it. This keeps the boundary rule in one place.
- After every successful move, check the pit and treasure coordinates before reading the next command. Ending the program from inside a `switch` inside a loop is easiest with a `bool` flag that you test in the loop condition, since `break` inside a `switch` only leaves the `switch`.
- Printing the direction name requires mapping the code back to a word — a second `switch` or a chain of `if-else` does this without needing an array.
- The final map does not require an array. Use a nested loop over y from 5 down to -5 and x from -5 up to 5, and decide what character to print at each coordinate using `if-else`.

Submission Checklist

Before you upload, confirm that you are submitting exactly these five files, and that each one compiles and runs on its own:

File Name	Question	Points
`kaprekar.cpp`	Kaprekar's Constant Chase	20
`wifiMap.cpp`	Wi-Fi Signal Map	20
`runLength.cpp`	Run-Length Stream Compressor	20
`snoozeClock.cpp`	Snooze Clock	20
`marsRover.cpp`	Mars Rover	20
	Total	**100**

- Each file contains your name, ERP ID, and section in a comment at the top.

- Each program compiles with `g++` with no errors.
- You have tested every program with the boundary cases described in its question.
- No program uses arrays, functions, recursion, pointers, strings, or STL containers.

— End of Homework 01 —